

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0405

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

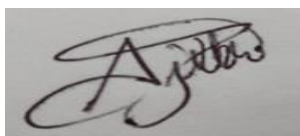
Duration: 30 minutes

Marks: 25

Code of Conduct:

I M. Ajitha (192525391)__(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



QUESTIONS:

Total Marks:25

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

- Identify the conceptual error in the likelihood formulation.
- Explain how this mistake affects parameter estimation.
- Suggest the correct mathematical formulation and reasoning.
- How would using log-likelihood fix computational issues?

a) Identify the conceptual error in the likelihood formulation.

Conceptual Error:

The student defined the likelihood as the sum of probabilities of all observations instead of the product of probabilities.

• **Incorrect likelihood:**

$$L(\theta) = \sum_{i=1}^n P(x_i | \theta) \quad L(\theta) = \sum_{i=1}^n P(x_i | \theta)$$

• **Correct likelihood:**

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta) \quad L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

Reason:

In MLE, each data sample is assumed to be independent. Therefore, the joint probability of all observations is obtained by multiplying their probabilities, not adding them.

b) **Explain how this mistake affects parameter estimation.**

Using the sum instead of the product causes:

- The likelihood no longer represents the joint probability of the dataset.
- The estimated parameters will not maximize the true likelihood.
- The classifier learns incorrect model parameters.
- Prediction accuracy decreases, even when a large amount of training data is available.

Result: The model gives poor parameter estimates and unreliable predictions.

c) **Suggest the correct mathematical formulation and reasoning.**

The correct Maximum Likelihood Estimation (MLE) is:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta) \quad L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

For a binary classifier:

$$L(\theta) = \prod_{i=1}^n P(y_i | x_i, \theta) \quad L(\theta) = \prod_{i=1}^n P(y_i | x_i, \theta)$$

Reasoning:

- Each observation contributes independently to the likelihood.
- Multiplying probabilities gives the probability of observing the entire dataset.
- The best parameter θ is the one that maximizes this likelihood.

d) **How would using log-likelihood fix computational issues?**

The log-likelihood is:

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta) \quad \log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

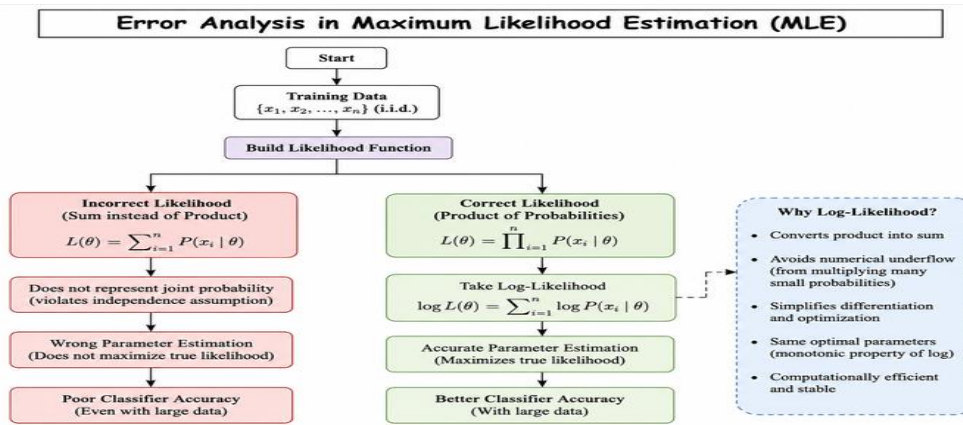
or for binary classification,

$$\log L(\theta) = \sum_{i=1}^n \log P(y_i | x_i, \theta) \quad \log L(\theta) = \sum_{i=1}^n \log P(y_i | x_i, \theta)$$

Advantages:

- Converts products into sums, making calculations easier.
- Prevents numerical underflow caused by multiplying many very small probabilities.
- Simplifies differentiation and optimization.
- Produces the same optimal parameter values because the logarithm is a monotonic increasing function.

Result: Log-likelihood makes MLE computationally efficient, numerically stable, and easier to optimize.



2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

a) List possible implementation errors in the learning algorithm.

- b) Analyze how incorrect learning rate or weight updates affect convergence.
- c) Identify dataset-related issues that may cause this problem.
- d) Propose modifications to ensure convergence.

a) List possible implementation errors in the learning algorithm.

Possible implementation errors are:

- Incorrect weight update formula.
- Bias term is not included or updated.
- Wrong prediction rule (using an incorrect threshold).
- Learning rate is set to an invalid value.
- Weights are not initialized properly.
- Labels are not encoded correctly (should be +1 and -1 or 1 and 0 consistently).

b) Analyze how incorrect learning rate or weight updates affect convergence.

- A very high learning rate causes large weight changes, making the model unstable and unable to converge.
- A very low learning rate results in very slow learning and requires many iterations.
- Incorrect weight update equations move the decision boundary in the wrong direction.
- As a result, the Perceptron keeps making classification errors even on a linearly separable dataset.

c) Identify dataset-related issues that may cause this problem.

Possible dataset-related issues include:

- Incorrect class labels in the training data.
- Presence of noisy or duplicate samples.
- Features are not properly normalized or scaled.
- Data preprocessing errors (missing or incorrect values).
- Although expected to be linearly separable, the dataset may actually contain overlapping classes due to labeling mistakes.

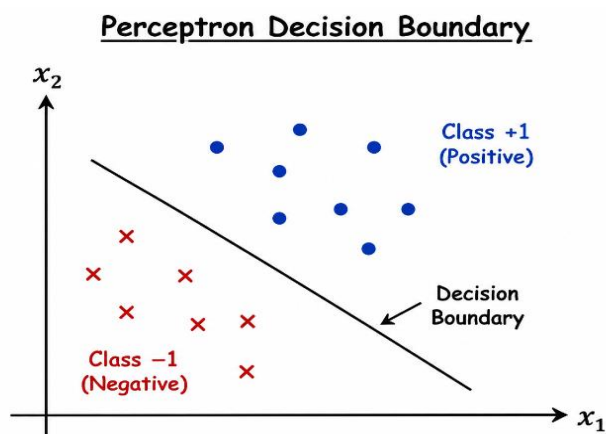
d) Propose modifications to ensure convergence.

To ensure convergence:

- Use the correct Perceptron weight update rule:

$$w = w + \eta(y - \hat{y})x$$
- Include and update the bias term.
- Choose a suitable learning rate (small positive value such as 0.01 or 0.1).
- Verify that class labels are encoded consistently.
- Normalize the input features.
- Remove incorrect or noisy training samples and verify that the dataset is linearly separable.

Result: With the correct implementation and a clean linearly separable dataset, the Perceptron converges to a valid decision boundary.



3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

- Identify possible mathematical or coding errors in gradient computation.
- Explain the role of activation functions in gradient flow.
- Analyze how vanishing or exploding gradients affect training.
- Suggest techniques to fix the issue (e.g., normalization, initialization).

a) Identify possible mathematical or coding errors in gradient computation.

Possible errors are:

- Incorrect calculation of gradients using the chain rule.
- Wrong derivative of the activation function.
- Incorrect weight or bias update equation.
- Wrong sign in gradient descent update (adding instead of subtracting the gradient).
- Errors in matrix dimensions or indexing during backpropagation.
- Learning rate set too high or too low.

b) Explain the role of activation functions in gradient flow.

- Activation functions introduce **non-linearity** into the network.
- During backpropagation, their derivatives determine how gradients are passed from one layer to another.
- Functions like Sigmoid and Tanh may produce very small gradients for large inputs.
- ReLU helps maintain stronger gradients for positive inputs, making training faster and more stable.

c) Analyze how vanishing or exploding gradients affect training.

- Vanishing Gradient:** Gradients become extremely small while moving backward through the network. As a result, early layers learn very slowly or stop learning completely.
- Exploding Gradient:** Gradients become very large, causing huge weight updates. This makes the training unstable, and the loss may increase instead of decreasing.

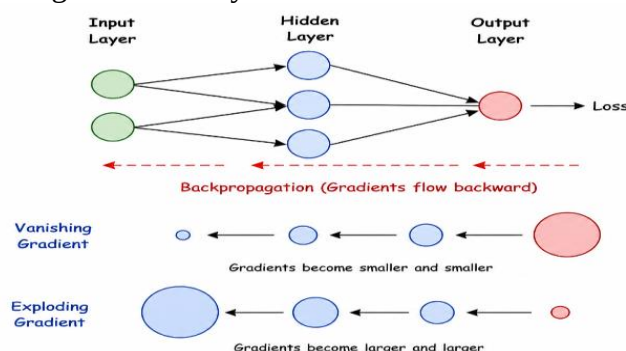
Result: Both problems prevent the model from learning the correct parameters.

d) Suggest techniques to fix the issue (e.g., normalization, initialization).

To improve training:

- Normalize the input data.
- Use proper weight initialization (Xavier or He initialization).
- Use ReLU or its variants instead of Sigmoid for deep networks.
- Apply Batch Normalization to stabilize training.
- Use Gradient Clipping to prevent exploding gradients.
- Choose an appropriate learning rate or use adaptive optimizers such as **Adam**.

Result: These techniques improve gradient flow, stabilize training, and help the model reduce loss and converge successfully.



4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

- Explain why batch gradient descent is slow in large datasets.
- Analyze the cause of fluctuations in SGD.
- Compare the error behavior of Mini-Batch Gradient Descent.
- Recommend the best approach and justify with error trade-offs.

a) Explain why Batch Gradient Descent is slow in large datasets.

- Batch Gradient Descent uses the entire training dataset to calculate the gradient in every iteration.
- Processing all samples takes more time and requires higher computational power.
- Parameter updates happen only after one complete pass through the dataset.
- Hence, training becomes slow for large datasets.

b) Analyze the cause of fluctuations in SGD.

- Stochastic Gradient Descent (SGD) updates the model using **one training sample at a time**.
- Since each sample is different, the calculated gradient varies at every update.
- This causes the loss to fluctuate instead of decreasing smoothly.
- Although noisy, these updates help the model learn faster.

c) Compare the error behavior of Mini-Batch Gradient Descent.

- Mini-Batch Gradient Descent updates the model using a **small group of samples** (mini-batch).
- The error curve is smoother than SGD but faster than Batch Gradient Descent.
- It reduces the noise in parameter updates while maintaining good learning speed.
- Therefore, it provides a good balance between stability and efficiency.

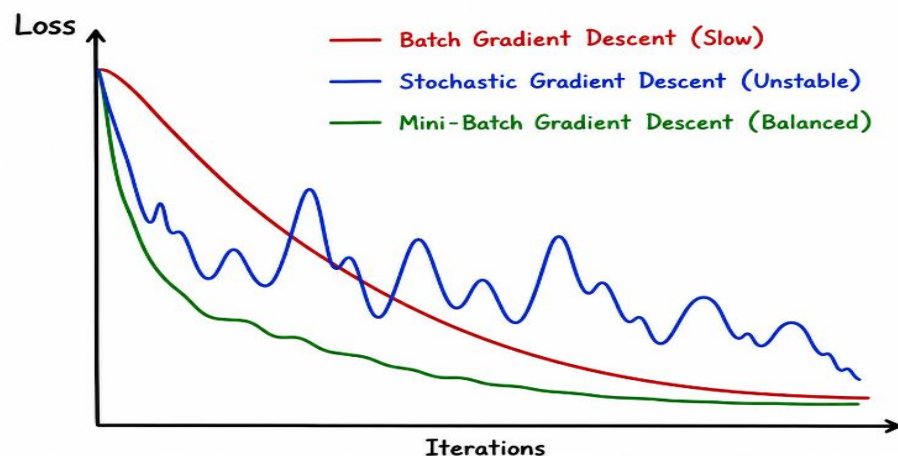
d) Recommend the best approach and justify with error trade-offs.

Recommended Approach: Mini-Batch Gradient Descent

Justification:

- Faster than Batch Gradient Descent.
- More stable than SGD with fewer loss fluctuations.
- Requires less memory than Batch Gradient Descent.
- Achieves faster convergence while maintaining good accuracy.
- Widely used in deep learning because it offers the best balance between speed and stable error reduction.

Result: Mini-Batch Gradient Descent is the preferred choice because it combines the advantages of both Batch Gradient Descent and Stochastic Gradient Descent, giving efficient and reliable model training.



5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

- Explain how high dimensionality increases model error.
- Identify signs of overfitting in this scenario.
- Analyze why distance-based learning fails in high dimensions.
- Suggest dimensionality reduction or regularization techniques to improve performance.

a) Explain how high dimensionality increases model error.

- High-dimensional data contains a large number of features.
- Many features may be irrelevant or redundant.
- The model becomes too complex and starts learning noise instead of useful patterns.
- As a result, it performs well on training data but gives higher error on unseen test data.

b) Identify signs of overfitting in this scenario.

The signs of overfitting are:

- Very high training accuracy.
- Low test accuracy.
- Large gap between training and testing performance.
- The model memorizes the training data instead of learning general patterns.

c) Analyze why distance-based learning fails in high dimensions.

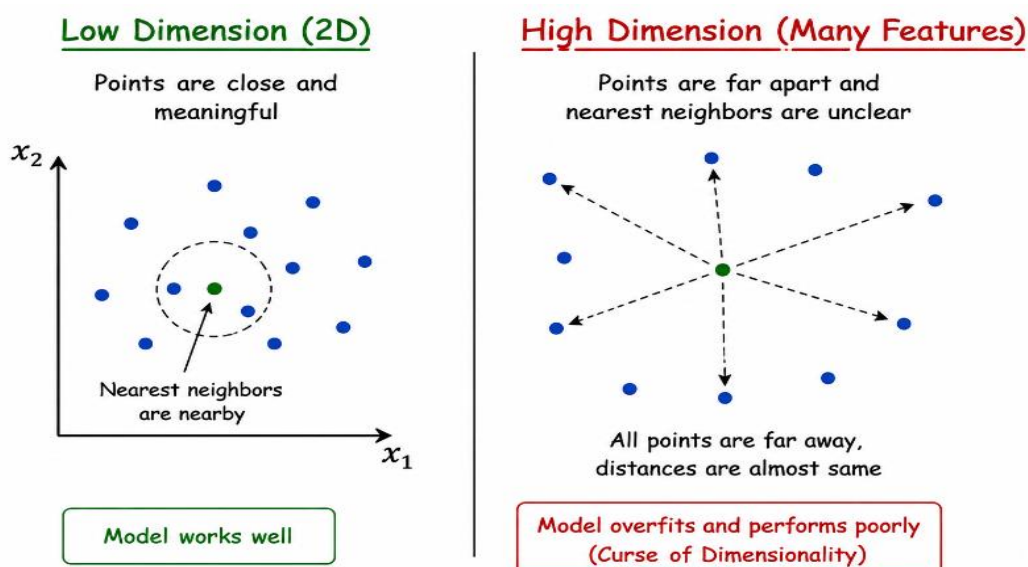
- In high-dimensional space, data points become very sparse.
- The distances between points become almost similar.
- It becomes difficult to identify the nearest neighbors correctly.
- Therefore, distance-based algorithms such as K-Nearest Neighbors (KNN) perform poorly.

d) Suggest dimensionality reduction or regularization techniques to improve performance.

To improve performance:

- Remove irrelevant or duplicate features.
- Apply Principal Component Analysis (PCA) to reduce the number of features.
- Use feature selection methods to keep only important features.
- Apply regularization techniques such as L1 (Lasso) or L2 (Ridge) to reduce overfitting.
- Increase the amount of training data whenever possible.

Result: Reducing unnecessary features and using regularization helps improve model accuracy and enables better performance on test data.



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving	20	Logical, well-structured, and	Mostly logical approach with	Basic approach with some	Illogical or unclear
Approach		efficient approach	minor gaps	inconsistencies	approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand